Web Application Penetration Testing Report

DVWA (Damn Vulnerable Web Application) — Local Assessment

Project	CloudExify Cybersecurity Internship — Month 2, Project 3
Assessment Type	Web Application Penetration Testing
Target Application	DVWA v1.10+ (localhost, XAMPP)
Security Level Tested	Low
Tools Used	Burp Suite Community Edition, Browser DevTools, XAMPP
Tester	Humayun
Assessment Date	August 17, 2026

Confidential — prepared for CloudExify internship review purposes only. All testing was performed exclusively against a locally hosted, intentionally vulnerable training application (DVWA) on the tester's own machine. No external or third-party systems were accessed.

1. Executive Summary

This report documents a penetration test performed against Damn Vulnerable Web Application (DVWA), hosted locally via XAMPP, as part of the CloudExify Month 2 cybersecurity internship curriculum. Testing was conducted at DVWA's **Low** security level using Burp Suite Community Edition and manual browser-based testing.

Five vulnerability classes from the OWASP Top 10 were tested and successfully exploited: SQL Injection, Reflected Cross-Site Scripting (XSS), Stored XSS, Cross-Site Request Forgery (CSRF), and Unrestricted File Upload leading to Remote Code Execution (RCE). All five attacks succeeded, confirming DVWA's Low security level provides no meaningful input validation, output encoding, or request-origin verification.

#	Vulnerability	OWASP Category	Result	Severity
1	SQL Injection (auth bypass + data extraction)	A03: Injection	Exploited	Critical
2	Reflected XSS	A03: Injection	Exploited	High
3	Stored XSS	A03: Injection	Exploited	High
4	Cross-Site Request Forgery (CSRF)	A01: Broken Access Control	Exploited	High
5	Unrestricted File Upload -> RCE	A04: Insecure Design	Exploited	Critical

2. Methodology

Testing followed a manual, black-box approach consistent with standard web application penetration testing methodology (OWASP Testing Guide):

• **Reconnaissance:** Identified DVWA's vulnerability modules via the application's built-in navigation menu.

• **Baseline testing:** Captured normal application behavior for each target function before injecting payloads.

• **Exploitation:** Used Burp Suite Repeater to intercept, modify, and resend HTTP requests with crafted payloads. Browser-based testing was used directly for XSS and CSRF where request manipulation was unnecessary.

• **Verification:** Confirmed each vulnerability's impact (data disclosure, script execution, unauthorized state change, or code execution) with screenshot evidence.

• **Reporting:** Documented findings, evidence, and remediation guidance per OWASP Top 10 categories.

2.1 Testing Environment

Component	Detail
Operating System	Windows (Desktop-HGJ5PAR)

Web Server	Apache 2.4.58 (XAMPP)
Database	MySQL (via XAMPP)
PHP Version	8.2.12
Target Application	DVWA — http://localhost/dvwa/
Proxy/Testing Tool	Burp Suite Community Edition v2026.7.3

3. Detailed Findings

3.1 SQL Injection — Authentication Bypass & Data Extraction

CRITICAL

Location: `/dvwa/vulnerabilities/sqli/?id=` (GET parameter, User ID search field)

The SQL Injection module accepts a numeric User ID and returns matching user records. The `id` parameter is concatenated directly into a SQL query without sanitization or parameterization, allowing arbitrary SQL logic to be injected.

Step 1 — Baseline request. A normal request for user ID 1 returns a single record:

```
GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: localhost
```

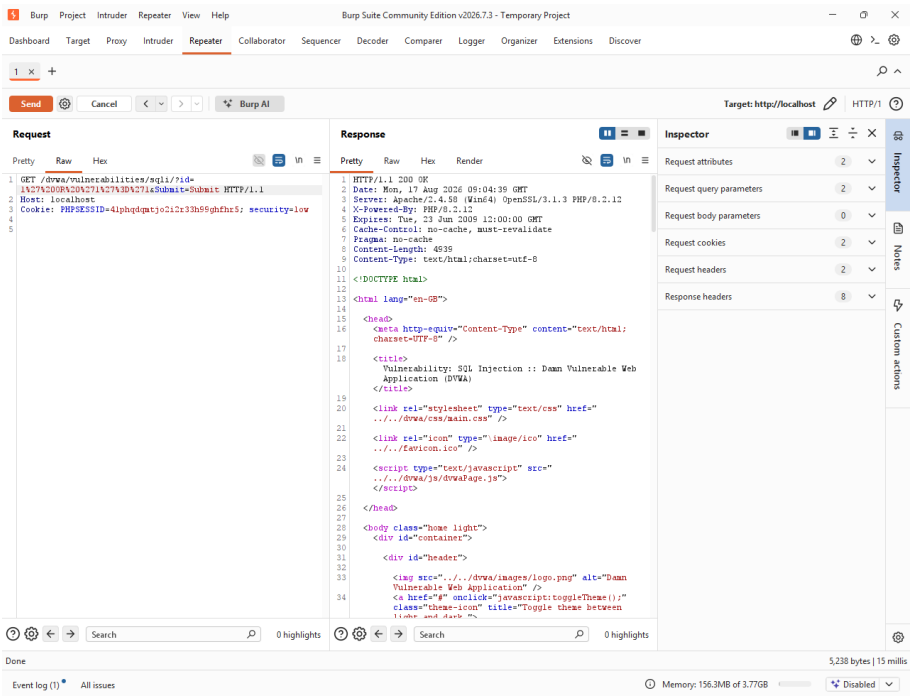

Figure 1 — Baseline response for id=1, returning a single user (admin).

Step 2 — Authentication bypass payload. Submitting `1' OR '1'='1` as the ID causes the WHERE clause to evaluate as always-true, returning every row in the users table instead of one:

```
id = 1' OR '1'='1
```

The response returned all five DVWA user records (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) from a query intended to return at most one.

Step 3 — Data extraction via UNION-based injection. A UNION SELECT payload was used to pull structured data — usernames and password hashes — directly from the `users` table:

```
id = 1' UNION SELECT user,password FROM users-- -
```

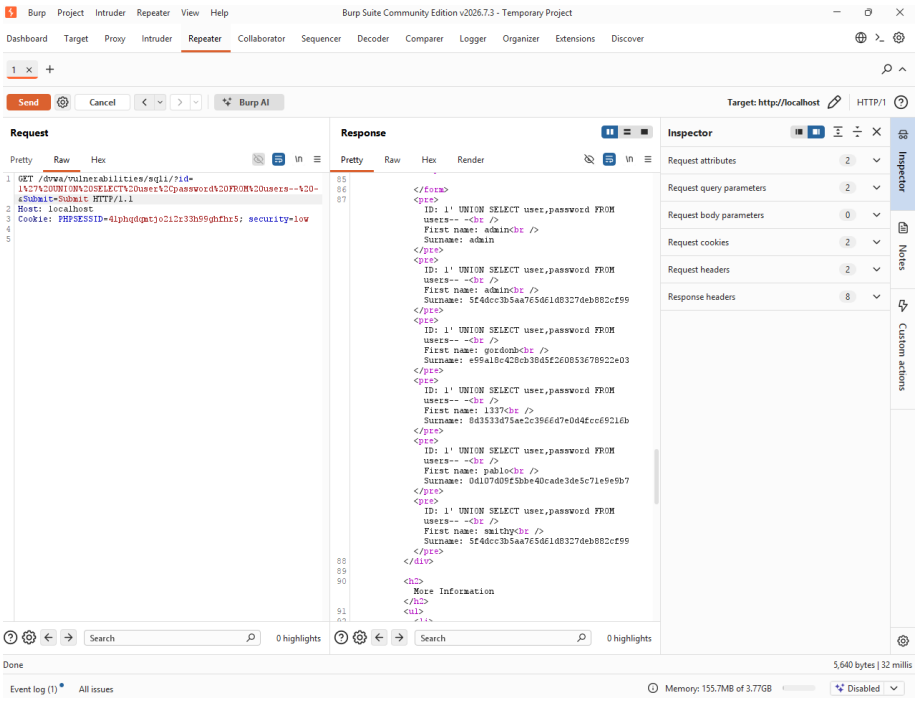

Figure 2 — Burp Suite Repeater response showing extracted usernames and MD5 password hashes.

Extracted credentials:

Username	Password Hash (MD5)
`admin`	`5f4dcc3b5aa765d61d8327deb882cf99`
`gordonb`	`e99a18c428cb38d5f260853678922e03`
`1337`	`8d3533d75ae2c3966d7e0d4fcc69216b`
`pablo`	`0d107d09f5bbe40cade3de5c71e9e9b7`
`smithy`	`5f4dcc3b5aa765d61d8327deb882cf99`

Notable observation: `admin` and `smithy` share the same password hash, indicating both accounts use the identical password — a password-reuse weakness worth flagging alongside the injection itself.

Impact: Complete bypass of query-level access control and full disclosure of all user credentials from the database, without any prior authentication beyond reaching the vulnerable page.

3.2 Reflected Cross-Site Scripting (XSS)

HIGH

Location: `/dvwa/vulnerabilities/xss_r/?name=` (GET parameter, name field)

The Reflected XSS module echoes the submitted `name` parameter directly back into the page HTML without encoding or sanitizing special characters, allowing injected script tags to execute in the victim's browser.

Payload submitted:

```
<script>alert('XSS')</script>
```

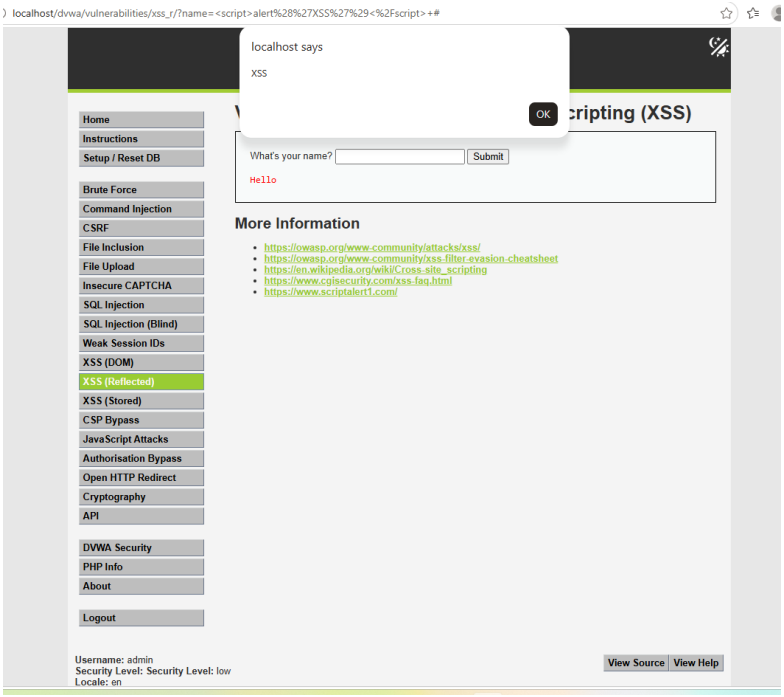

Figure 3 — Injected script executes immediately, popping a JavaScript alert box.

Impact: An attacker could craft a malicious link containing this payload and send it to a victim; if clicked while authenticated, arbitrary JavaScript would run in the victim's session — enabling session/cookie theft, page defacement, or redirection to phishing pages.

3.3 Stored Cross-Site Scripting (XSS)

HIGH

Location: `/dvwa/vulnerabilities/xss_s/` (Guestbook Name/Message fields)

The Stored XSS module (a guestbook feature) saves submitted messages to the database and renders them, unescaped, to every visitor of the page — unlike reflected XSS, this payload persists and affects all future viewers, not just the submitter.

Payload submitted as the guestbook message:

```
<script>alert('Stored XSS')</script>
```

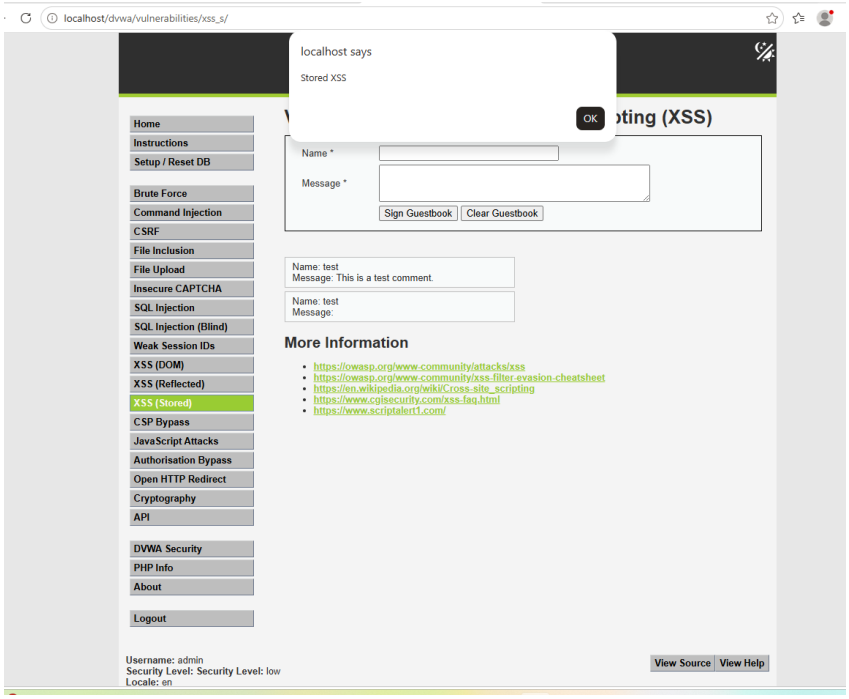

Figure 4 — Alert fires immediately upon submitting the guestbook entry.

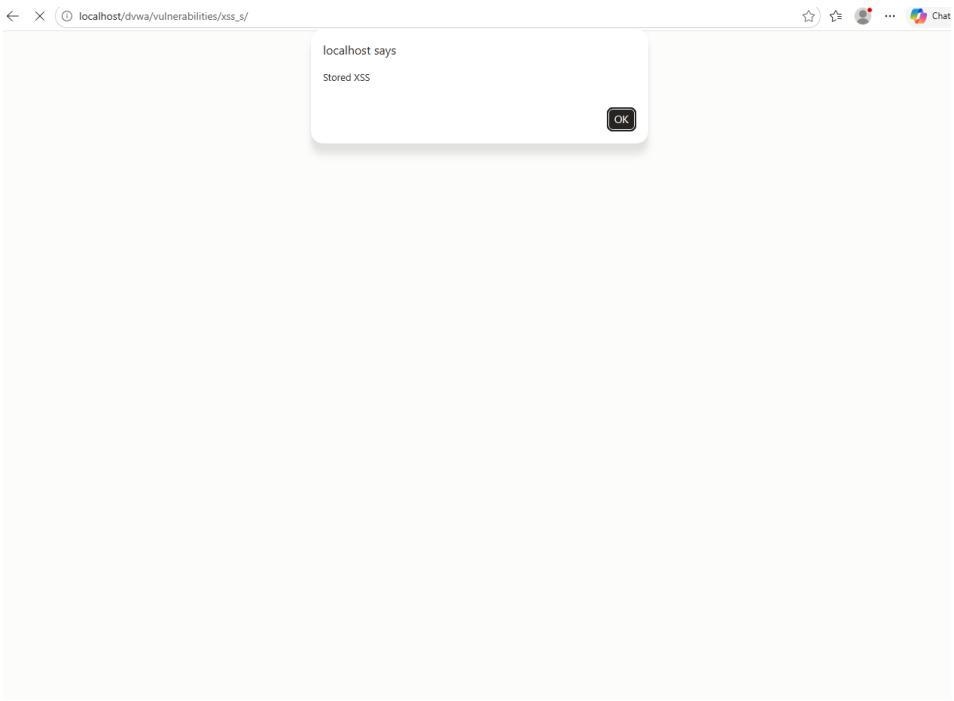

Figure 5 — Alert fires again on a fresh page load, confirming the payload is persisted server-side and executes for every visitor, not just the original submitter.

Impact: Significantly more severe than reflected XSS — the payload does not require tricking a victim into clicking a crafted link. Every user who simply views the guestbook page is affected automatically, making this suitable for mass credential/session theft or worm-like propagation.

3.4 Cross-Site Request Forgery (CSRF)

HIGH

Location: `/dvwa/vulnerabilities/csrf/` (Change Password form)

The password-change form accepts `password_new` and `password_conf` parameters via a simple GET request, with no anti-CSRF token, no re-authentication step, and no verification that the request originated from DVWA's own form.

A proof-of-concept HTML page was built containing a hidden image tag pointing at the password-change endpoint with attacker-chosen values:

```
<img src="http://localhost/dvwa/vulnerabilities/csrf/?password_new=hacked123&password_conf=hacked123&Change=Change" width="0" height="0">
```

When this page was opened in a browser tab holding an active, authenticated DVWA session, the hidden request fired automatically — silently changing the admin account's password with zero user interaction beyond opening the page. Logging in afterward with the new password (`hacked123`) succeeded, confirming the change took effect.

Testing note: initially hosting the proof-of-concept page from the local filesystem (`file://`) failed to trigger the exploit, because modern browsers withhold session cookies from cross-origin requests under default SameSite cookie policy. Serving the same file from `http://localhost/` (same-site as DVWA) resolved this — a detail specific to modern browser defenses rather than any protection implemented by DVWA itself, which has none.

Impact: Any authenticated user who visits an attacker-controlled page (or is served a malicious ad/embed) could have their account password silently changed, leading to full account takeover.

3.5 Unrestricted File Upload — Remote Code Execution

CRITICAL

Location: `/dvwa/vulnerabilities/upload/` (file upload form)

The File Upload module is intended to accept images, but performs no validation of file type, extension, or content on the server side. A PHP web shell was uploaded directly.

Malicious file contents (saved as shell.php):

```
<?php echo shell_exec($_GET['cmd']); ?>
```

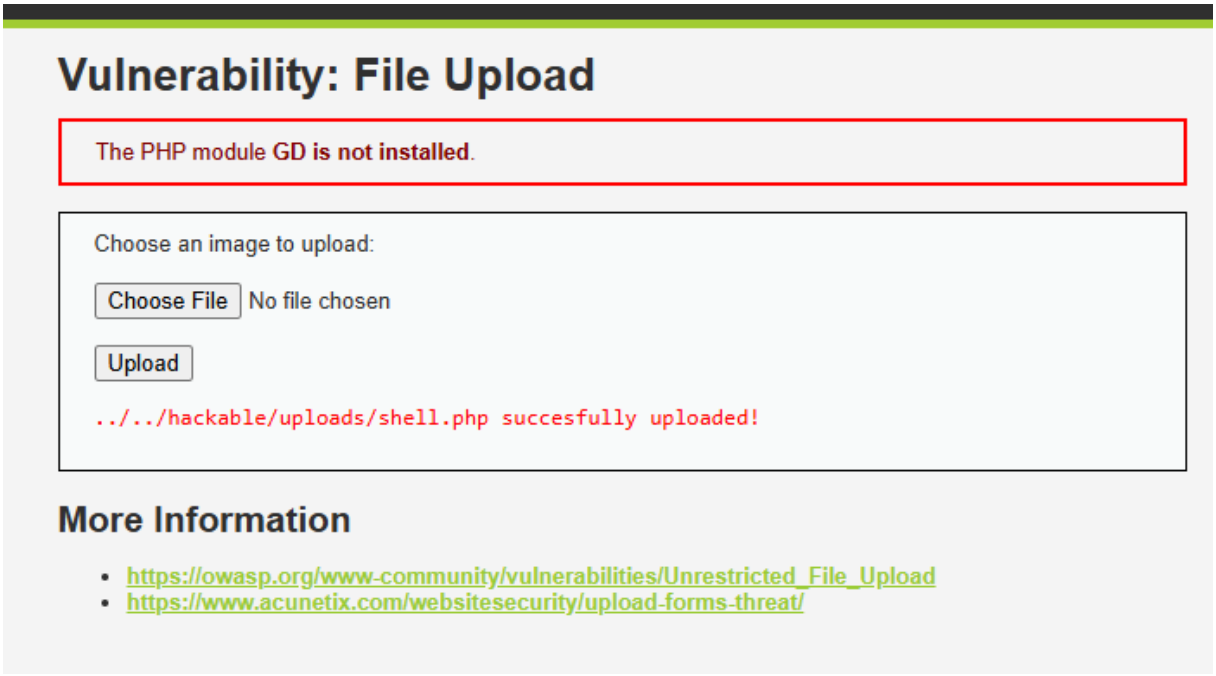

Figure 6 — DVWA accepts the .php file with no type restriction and confirms upload to /hackable/uploads/shell.php.

The uploaded shell was then invoked directly via URL with an attacker-supplied command:

```
GET /dvwa/hackable/uploads/shell.php?cmd=whoami
```

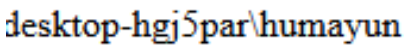

Figure 7 — The shell executes the whoami command, returning the server's Windows account (desktop-hgj5par\humayun) — confirming arbitrary command execution.

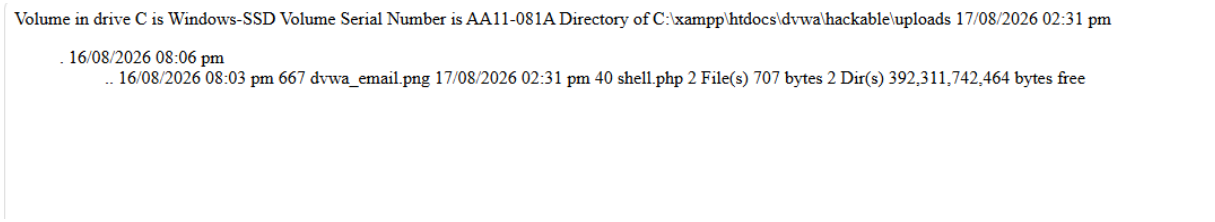

Figure 8 — A follow-up dir command lists the contents of the uploads directory, further confirming full shell functionality.

Impact: This is the most severe finding in the assessment. An attacker can execute arbitrary operating-system commands on the underlying server with the privileges of the web server process — enabling full server compromise, data theft, lateral movement, or use of the server as a pivot point for further attacks.

4. Remediation Guidance

Vulnerability	Recommended Fix
SQL Injection	Use parameterized queries / prepared statements for all database access; never concatenate user inp
Reflected XSS	HTML-encode all user-supplied data before rendering it into a page (e.g. htmlspecialchars() in PHP). U
Stored XSS	Apply the same output encoding on all data retrieved from the database before rendering, since stored
CSRF	Implement per-session, per-form anti-CSRF tokens validated server-side on every state-changing requ
File Upload / RCE	Validate file type by content (MIME sniffing / magic bytes), not filename extension. Store uploads outsi

5. Conclusion

All five tested vulnerability classes were successfully exploited against DVWA at its Low security setting, confirming the application performs no meaningful input validation, output encoding, or request-origin verification at this level. The most severe finding — unrestricted file upload leading to remote code execution — would, in a production context, constitute full compromise of the underlying server. The SQL injection finding similarly allowed complete extraction of user credentials with no prior authentication.

These results align with DVWA's purpose as a deliberately vulnerable training platform and illustrate, in practice, why the remediation techniques above (parameterized queries, output encoding, anti-CSRF tokens, and strict server-side upload validation) are considered baseline requirements for production web applications.

6. Appendix — Environment Verification

XAMPP Control Panel confirming Apache and MySQL services active throughout testing:

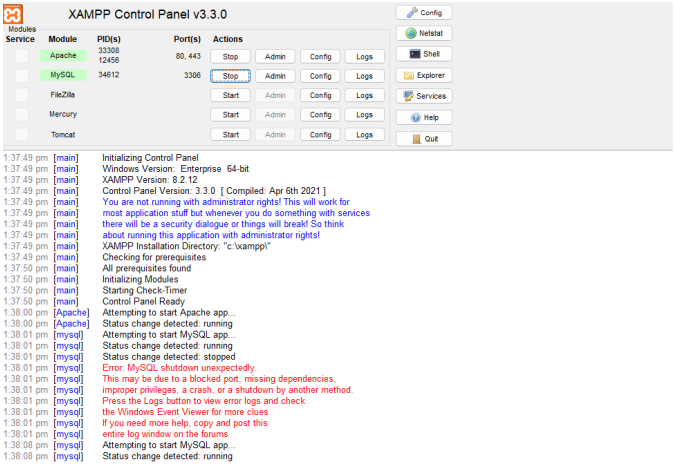

Figure 9 — XAMPP Control Panel, Apache and MySQL running.